MOBILE DJ / PRODUCT BLUEPRINT

DOCUMENT 17 OF 19

Open-source and home-lab guide

A new touch-first performance instrument.
Professional audio, liquid glass and mechanical craft.



DESIGN HANDOFF BASELINE - PLANNING ONLY

29 October 2026 | Crystal Current selected; S26 Ultra test target

Independent of the previous Mixline preview. Targets are not measured results.

Version: 1.0 **Date:** 29 October 2026 **Status:** Design handoff baseline; local delivery first, with AWS and optional publication deferred **Confirmed intent:** Personal local app first; potential GitHub open-source sharing; later optional AWS extension and D-versus-AWS comparison; no local host selected; approximately USD 35/month future AWS planning target excluding ParDay AI-provider charges **Owners:** Architecture, Integrations, Release/Security, Documentation and product owner **Companions:** Documents 05, 09, 10, 11, 13, 16 and 18; decision ledger; acquisition, AWS and portable deployment contracts

1. What this decision changes

Build the product for the owner's personal workflow first. Preserve the selected Crystal Current interface, Galaxy S26 Ultra target, explicitly armed ParDay assistance and complete professional feature scope. Personal distribution does not lower the audio, recovery, privacy or source-identity requirements.

The latest answer selects the phone-native local version first and AWS expansion later. No local host, service account or cloud worker is required to start design or deliver the first local app. Document 18 owns the deferred cloud proposal and the approximate USD 35/month planning target, excluding ParDay AI-provider charges. Account, region, instance and deployment remain open; the target is not an approved resource purchase or guaranteed bill. Local LAN/remote profiles in this guide remain optional portability references.

Plan software others can inspect, build and operate with their own equipment and credentials. This does not commit the owner to public hosting, subsidized AI or support for every phone.

Open-source publication is a planning direction. Licence, GitHub account/name and publication remain unselected. Prepare an inspectable release package for that decision. This document creates no repository, public release, container image, hosted endpoint or licence grant.

2. Personal project and open source are different decisions

Personal use describes the owner's goal and maintenance commitment. An open-source licence gives downstream users permissions defined by that licence. The Open Source Definition requires redistribution and modification rights and does not allow a restriction excluding business use. A custom personal-use-only or noncommercial clause would therefore conflict with describing that licence as open source. [H1]

This distinction need not change the owner's intention. The README can describe a personal project, supported configurations and limited maintenance without adding incompatible usage restrictions. If the owner wants a binding noncommercial restriction, record that as a different source-available licensing choice rather than silently applying it.

A public GitHub repository alone is not sufficient licensing guidance. GitHub recommends an explicit licence so others know their rights to use, modify and distribute the work; its platform viewing/forking behavior does not replace a project licence. [H2]

3. Licence candidates and recommendation

The following is a decision aid. No option is selected or applied to the code in this planning pack.

Candidate	Relevant effect	Fit to evaluate
GPL-3.0-or-later	Copyleft conditions apply when covered software is conveyed; distributed object code needs a compliant corresponding-source route. Private use does not itself require public posting. [H3]	Leading candidate for the Android application if GPL components are included and shared improvements are preferred.

Candidate	Relevant effect	Fit to evaluate
AGPL-3.0-or-later	Adds a source-offer requirement for users interacting remotely with a modified covered network program. [H4]	Consider for a server only if the owner wants that network-use obligation; self-hosting alone is not a reason to select it automatically.
Apache-2.0	Permissive reuse with explicit patent provisions and licence/notice/change obligations. [H5]	Candidate for independently licensable first-party components when broader reuse matters and the dependency combination permits it.
MIT	Short permissive licence retaining its copyright/permission notice and warranty disclaimer. [H6]	Simple alternative for independently licensable components; it does not erase obligations from included dependencies.

The provisional recommendation is to evaluate GPL-3.0-or-later for the app, then decide the service licence after selecting its components and the desired contribution model. A single coherent first-party licence may be easier to maintain than several unnecessary licence boundaries. Do not adopt AGPL, a contributor agreement or a dual-licensing programme merely because they are available.

NewPipeExtractor explicitly uses GPL version 3 or later. Its inclusion would require a compatible distribution plan for the resulting application; an MIT label on our own files is not a blanket override. [H7] Review paid DSP, plugin components and native libraries against the actual combined work before promising a redistributable build.

yt-dlp illustrates why the exact artifact matters: its project source is under the Unlicense, while some bundled executables contain GPLv3+ components and other distributions include additional licences. [H8] Record the actual package, version, build flags and dependencies that are shipped. A Docker image, process boundary or HTTP API does not automatically settle whether components form one covered work or remove redistribution obligations. Review the actual integration rather than using architecture as a licensing shortcut.

4. Repository contents people can actually use

Plan one repository with clear component boundaries initially. Split repositories only if independent release or ownership needs justify it.

Planned area	Required content
Android app	Kotlin/Compose source, resource sources, build configuration and documented device requirements
Audio engine	C++ source, adapter interfaces, test harnesses and explicit real-time invariants
Acquisition module	Native on-device adapters first; shared job/provenance contracts and a replaceable boundary for a later optional service
Deployment	Local build and install instructions first; reproducible service/container setup, health checks and rollback only when an optional service is implemented
Documentation	Architecture, local build, personal installation, home-lab setup, source capabilities and troubleshooting
Contracts	App/service compatibility version, job lifecycle, cancellation, error taxonomy and migration rules

Planned area	Required content
Fixtures	Small original or redistribution-cleared media, fixture manifests and expected outcomes; no personal library dump
Release materials	Dependency inventory, licence texts/notices, source/release mapping, checksums and known limitations
Contribution materials	Issue templates, contribution guide, security reporting route and device-result template

Configuration examples contain placeholders and explained defaults. They must not require the owner's private server, signing key, account or music collection to complete a clean build. Installation documentation should distinguish a developer build, a personally signed build and any later project release APK. A working build from a fresh checkout is the standard, not the existence of a repository screenshot.

5. Code, assets and private state

Maintain separate records for source code, fonts/icons/textures, demonstration music, sample packs, model weights and third-party libraries. The top-level code licence should state its scope and point to the asset/dependency inventory. Preserve the licence and provenance of each included item. If an asset cannot be redistributed, provide a documented replacement or user-supplied configuration rather than an unexplained missing file.

Keep personal tracks, downloaded media, recordings, playlists containing private information, ParDay transcripts, provider tokens, API keys, cookies, server credentials and signing material out of public commits and release archives. Use a small deliberately shareable demo crate. Inspect release artifacts as well as source history; a clean working tree does not establish that an earlier commit or compiled resource contains no private material.

The repository should explain which components require separate external services or credentials. Each operator supplies their own credentials. Do not ship a shared paid API key, a bundled session from a music website, or a public endpoint funded by the owner as an undocumented dependency.

6. Home-lab operating model

Deliver the phone-native app first; the phone owns its performance library, preparation, acquisition job state and live engine. If a service is added later, offer a documented single-operator deployment before considering multi-tenant hosting. That optional service may resolve/acquire/process media or provide a separately configured ParDay gateway. AWS is the accepted later server option; this section preserves physical home-lab portability. Loss of any service leaves already prepared local performance usable.

Keep acquisition and AI capabilities separable in setup. A user may want an acquisition worker but no cloud model, or ParDay with their own API account while choosing on-device acquisition. The endpoint picker must state which server receives links or permitted context. A service address is not evidence that its operator is the app's publisher.

Document host OS/architecture, measured CPU/RAM/storage needs, runtime, persistent directories, temporary-media limits, credentials, reachability and pairing. Hardware minimums require comparison evidence; generic NAS or Raspberry Pi support is not assumed.

For a local home-lab profile, default to private LAN/VPN deployment with authenticated encrypted transport, per-device pairing and revocation. Private network reachability is not authentication. Pairing should use an expiring code or equivalent reviewed mechanism and a revocable credential; health/version negotiation precedes jobs. Never disable certificate validation or use trust-all TLS.

The phone may connect to its explicitly paired private home-lab endpoint. This control-plane trust does not let the backend fetch arbitrary private, loopback, link-local or metadata-service URLs as media. Keep the source-fetch SSRF rules in Document 16 separate from the configured service address. Container workers must not require privileged mode, the Docker socket or a full-home-directory mount; provide only bounded staging and necessary data paths. Prepared local DJ playback must remain independent of this service.

The runbook must explain where configuration, job state, temporary files, completed artifacts and logs live; which items are backed up; how expiration/deletion works; and how to restore without replaying completed jobs. Use bounded concurrency and per-job limits. Backup/restore should preserve user preparation while avoiding accidental publication of private media or secrets.

7. Deferred comparison of on-device and self-hosted acquisition

The owner selected local delivery before AWS expansion, while preserving comparison evidence for the later extension choice. Build and validate D first and retain its fixture/measurement records. When AWS is evaluated, compare D and AWS over recorded real phone networks; physical LAN and remote home-lab arms are optional portability work. Both candidates use the same native intake, selection, queue and library-publishing contract. The comparison is not a first-local-release gate, and one local demonstration does not establish the cloud candidate's infeasibility or sustained performance.

Use identical suitable source fixtures and selected formats, control the phone's network conditions, and apply the same resulting-media checks. Record the different source access paths rather than pretending AWS and phone egress are the same network. Record cold/warm resolution time, transfer and processing time, time until playable, completion/failure rate, battery change, thermal state, app memory, server resources, storage use and operator steps. Run acquisition alongside the declared DJ workload and test offline server loss, cancellation and recovery.

Report exact phone software, audio route, server hardware, adapter/runtime versions and output hashes. Separate source outages from implementation faults. Record per-format results rather than declaring a universal winner. A later hybrid may be justified by evidence, but the first local release does not need two complete implementations. Document 16 owns the acquisition semantics; Document 09 owns audio/device gates.

8. Updates and community operation

Version the Android/service protocol and advertise compatible ranges. A newer server must not reinterpret an old client's action silently. Pin extractor and processing dependencies in releases, test changes on the fixture corpus, and retain the preceding working deployment for rollback. Never replace a live audio component mid-set.

Document source outages, temporary adapter disablement and upstream changes without blaming every failure on a user's setup. An unsupported device report should collect the necessary profile and logs with private values redacted. Publish measured supported combinations; community reports are useful evidence but are not automatically official certification.

Require reproducible issues, focused changes and compatible contributions; credit authors. Decide whether contributions are accepted before promising review/support. Security reports need a real private route when the repository exists; do not present a placeholder address as monitored.

9. Publication readiness

Before public release, present the licence/notices, clean-build evidence, shareable fixtures, dependency inventory, source-to-binary mapping, tested setup and limitations. The owner then reviews the exact publication package. The planning guide does not substitute for a tested deployment runbook.

The personal app remains useful before that step. Google Play submission is not a gate for the selected personal-tool milestone; public store distribution can be reconsidered only as a new decision. Applicable Android installation/signing behavior, component licences and source-service requirements still need their own current review. Open-sourcing code or self-hosting a worker does not grant rights in a music catalog or turn an unverified source adapter into a supported one.

10. Acceptance evidence

The companion home_lab_deployment_profile.json records each case's applicability for the portable profile. Document 18 and aws_deployment_profile.json add the AWS-specific acceptance cases. All are unexecuted. App build reproduction, implemented-runtime safety and local/AI independence remain required. Service-specific variants apply only when a later service profile is evaluated or selected; neither they nor the D-versus-AWS comparison are local design/release prerequisites. HL-10 applies only before optional public publication.

ID	Required demonstration
HL-01	Clean-host reproduction: build the app and selected service profile using documented prerequisites, permitted dependencies and shareable fixtures.
HL-02	Pair and revoke: a new device pairs deliberately; revocation blocks subsequent service actions without deleting its local preparation.
HL-03	Authentication and TLS: reject untrusted clients/certificates; paired private control-plane access does not permit private-target media fetching.
HL-04	Server loss: prepared local mixing continues; unavailable acquisition reports its actual state without surprise playback or lost preparation.
HL-05	Resource limits: exhausted disk, hung/crashed workers and oversized jobs stay bounded and do not compromise live audio.
HL-06	Credential isolation: logs, artifacts and one worker cannot reveal another component's secrets or private context.
HL-07	Upgrade/recovery: protocol mismatch, rollback and backup restore preserve correct jobs and do not replay completed actions.
HL-08	AI independence: acquisition can run without ParDay; the local engine remains usable with all external services unavailable.
HL-09	Artifact cleanup: successful transfer, expiry, cancellation and failure follow the documented retention/deletion policy.
HL-10	Optional publication: review the exact source/history/artifacts, licence obligations and build mapping; exclude credentials, personal music and signing keys.

No build, deployment, licence adoption or public publication has been performed in this planning revision.

Sources checked 29 October 2026

- [H1] Open Source Initiative, Open Source Definition: https://opensource.org/osd

- [H2] GitHub, Licensing a repository: https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/licensing-a-repository

- [H3] GNU GPL v3 text, Open Source Initiative copy: https://opensource.org/license/gpl-3.0 ; steward version: https://www.gnu.org/licenses/gpl-3.0.en.html

- [H4] GNU AGPL v3 text, section 13, Open Source Initiative copy: https://opensource.org/license/agpl-3.0

- [H5] Apache Software Foundation, Apache License 2.0: https://www.apache.org/licenses/LICENSE-2.0

- [H6] MIT licence text: https://opensource.org/license/mit

- [H7] NewPipeExtractor, licence and integration documentation: https://github.com/TeamNewPipe/NewPipeExtractor

- [H8] yt-dlp, licensing and distribution documentation: https://github.com/yt-dlp/yt-dlp

Repository structure, deployment defaults, comparison method and acceptance criteria are proposed project decisions. The licence discussion is conditional on the actual components and distribution; it is not a claim that any candidate combination has already been cleared.